

5.5 程序中断方式

一、I/O 中断的产生

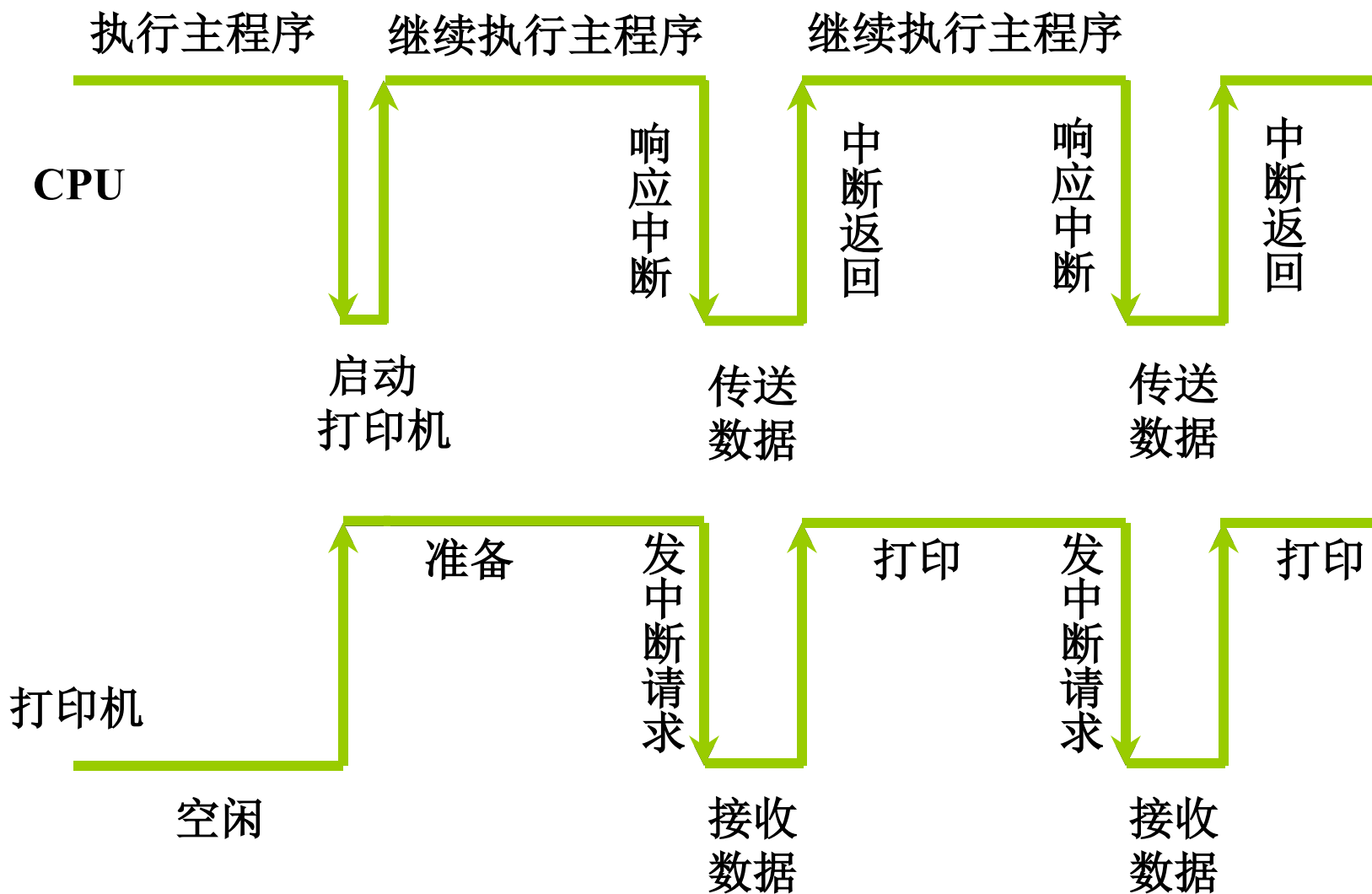
- ✓ 由前面内容可知：I/O设备与主机交换信息时，由于外设速度低于CPU，CPU启动设备后，需要等待一段时间后才能交换信息
- ✓ 在外设做准备时，CPU继续执行现行政程序，在外设准备就绪并提出中断请求之后，再暂时中断现行政程序，转入执行I/O服务程序，这就产生了I/O中断



但是，外设请求并非引发中断的唯一原因

以打印机为例

CPU 与打印机并行工作



二、中断的概念

中断可用于响应外设请求、控制I/O设备和主机传送信息，但其作用不是仅止于此

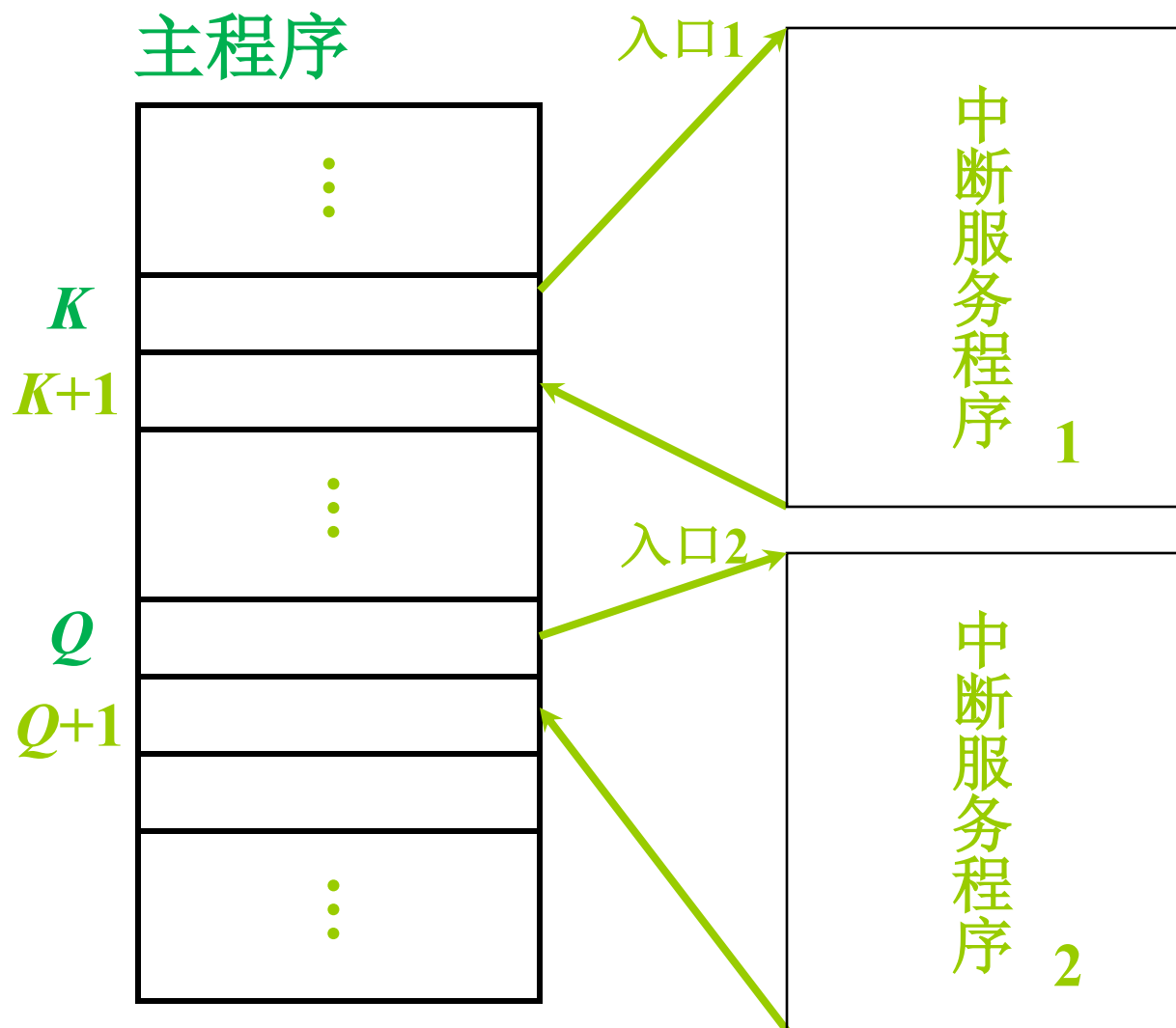
中断：计算机执行程序过程中，若出现异常情况或者特殊请求，则计算机转向对异常情况或者特殊请求的处理，这通过运行中断服务程序实现，处理结束后再返回到原程序的间断处，继续执行原程序

作用：中断机制提高计算机系统的效率和效能

实现中断功能所需要的软硬件技术统称为中断技术



概念：断点地址、入口地址



采用中断机制原因

- 低速外设就绪后提出中断请求
 - 应付突发事件的需要
 - 实时控制中响应外来信号、可靠性等需要
 - 软件中断指令引起的中断
- 后三者在后面章介绍

中断源：能引起中断的各种因素，例如I/O设备、以及各种突发事件

为了实现中断，计算机系统中必须配有相应的中断系统或者中断机构

为了处理I/O中断，在I/O接口电路中必须配置相应的硬件电路，包括中断请求触发器、屏蔽触发器、排队器、中断向量地址形成部件等



三、程序中断方式的接口电路

1. 配置中断请求触发器和中断屏蔽触发器

每台外设配置一个**中断请求触发器INTR**，其值为1时表示设备向CPU提出中断请求，只有设备就绪时才能提出中断请求，设备就绪的标识是完成触发器D状态为1

在I/O接口中需要设计一个**屏蔽触发器MASK**，当其为1时，表示封锁**中断源**的请求

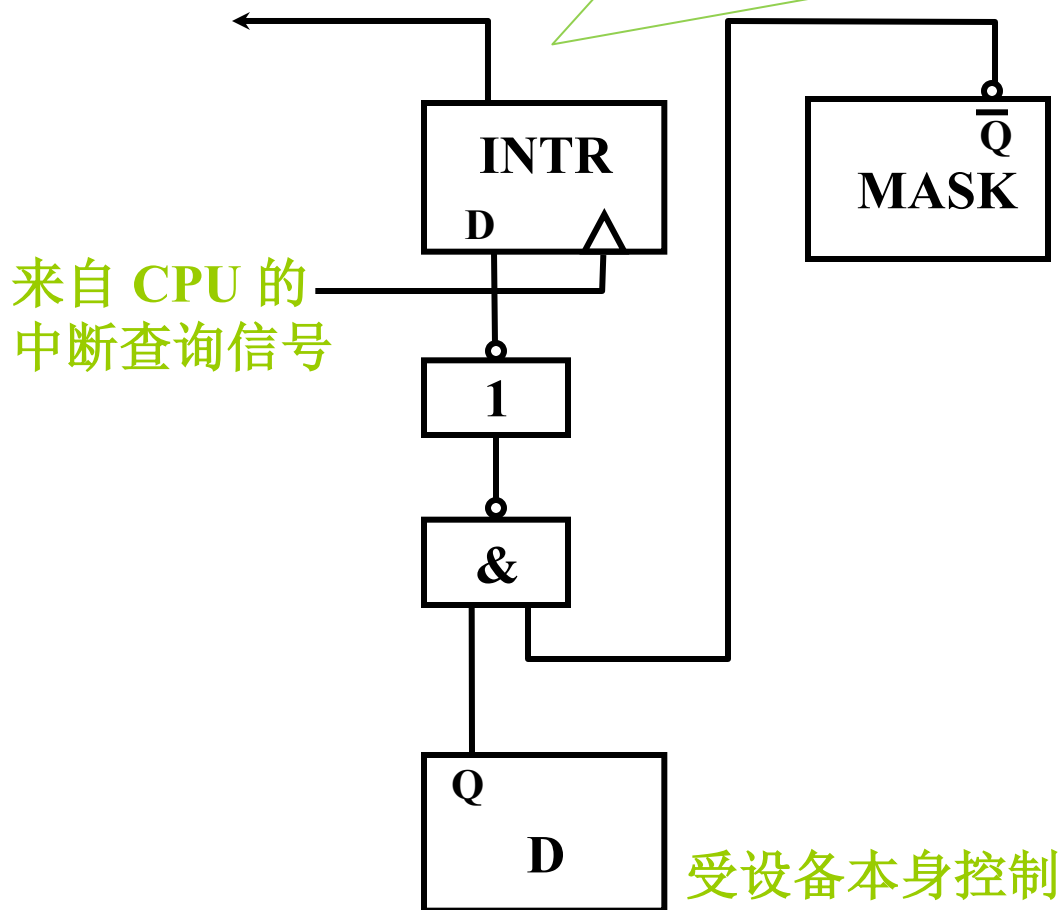
CPU在**每条指令**执行的最后阶段，**查询所有设备**中是否有中断请求，**若此时设备就绪 ($D=1$) 且未被屏蔽 ($MASK=0$)**，则中断查询信号将**INTR置1**



接口电路中的D、INTR、MASK触发器及中断查询信号：设备就绪（D=1）且未被屏蔽（MASK=0），则中断查询信号将INTR置1（懂得逻辑即可，电路不用记）

中断请求

D触发器，时钟信号到来时，输出跟随输入



INTR 中断请求触发器

INTR = 1 有请求

MASK

中断屏蔽触发器

MASK = 1 被屏蔽

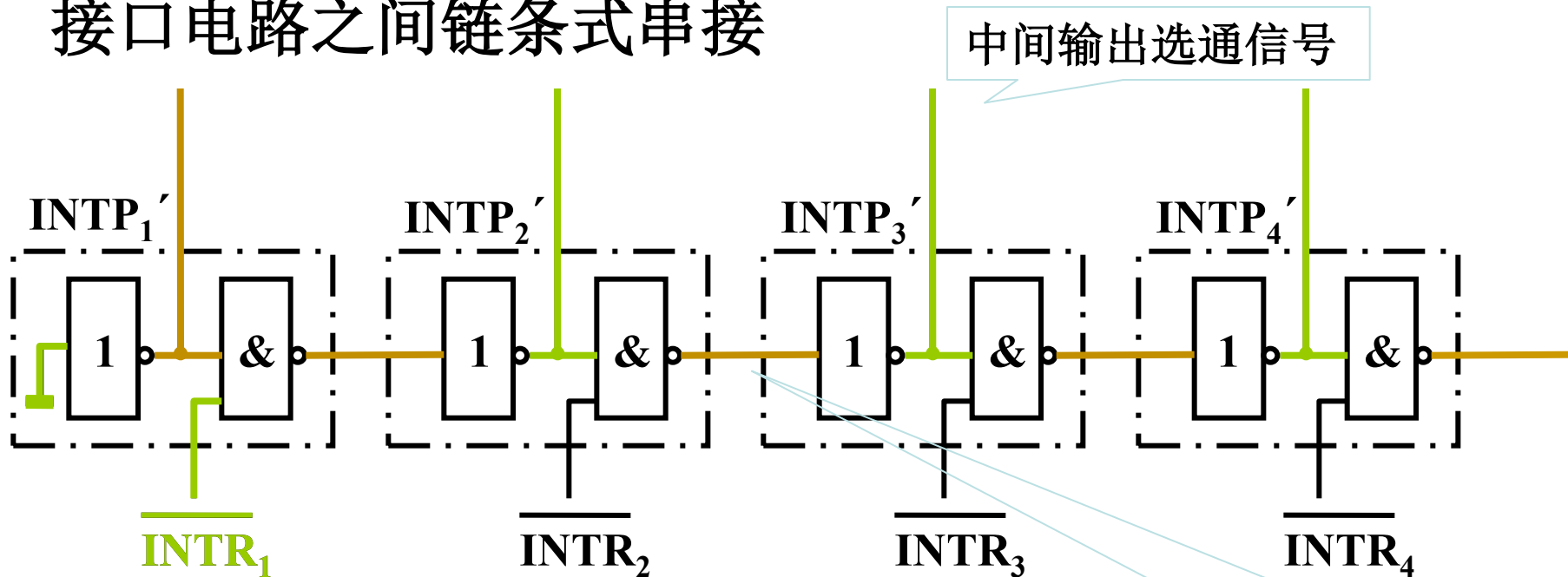
D 完成触发器

2. 排队器

- 多个中断源同时向**CPU**提出请求时，按照优先级高低予以响应，因此需要对中断源按照不同性质进行安排，赋予不同的优先级
- 对于I/O设备来说，为防止信息丢失，速度高的设备中断优先级也高
- 处理设备优先级可以采用硬件方法或者软件方法（后续内容），硬件方法包括：
 - 1) 在**CPU**中设置一个统一的硬件排队器，对所有中断源进行排队--后续内容；
 - 2) 在接口电路内分别设置各个设备的排队器--本章内容

链式排队器

下图为链式排队器的关键部分，每个接口内有一个反相器（是否受上级封锁）和一个与非门（是否封锁下级），各接口电路之间链条式串接



✓ 设备 1[#]、2[#]、3[#]、4[#] 优先级按 降序排列

$INTR_i = 1$ 有请求 即 $\overline{INTR_i} = 0$

某个接口向下一个接口输出1，则封住后面所有接口

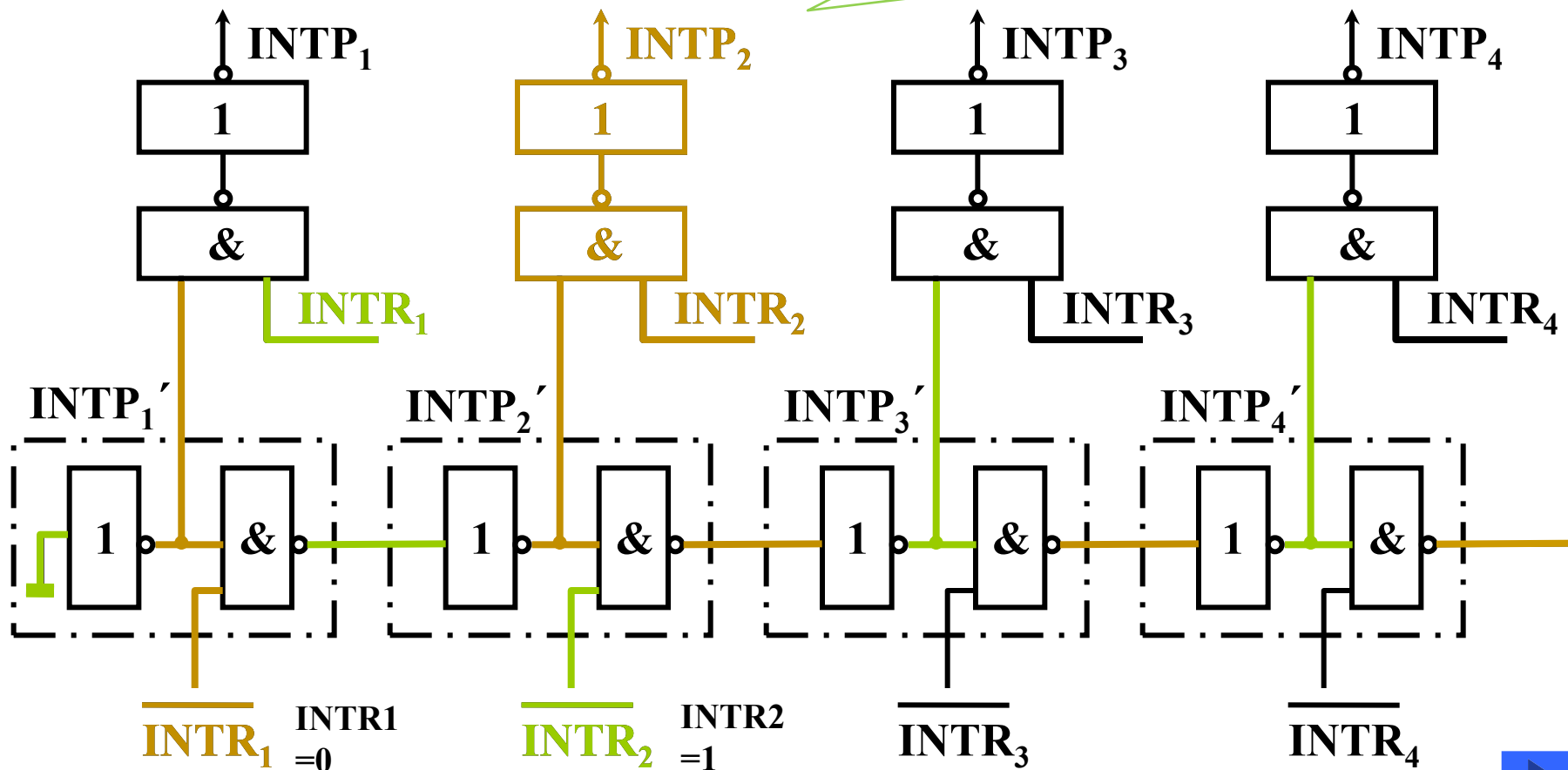
排队器工作原理

- 中间输出 $INTP_i'$ 是**选通信号**，决定本 $INTR$ 能否送出
- 当各中断源都没有提出中断请求时，各个中间输出端 $INTP_i'$ 都是高电平
- 若某个中断源提出中断请求，就迫使所有比其优先级低的中断源对应的 $INTP_i'$ 变为低电平（关门）
- 中间输出和后续电路一起工作，使得无论哪个（或多个）中断源提出中断请求，排队器的中间输出端只对一个为高电平（开门），相应地：**总输出端只有一个 $INTP$ 是高电平，这个就是最高优先级的中断请求信号**

掌握这个逻辑即可

排队器工作原理

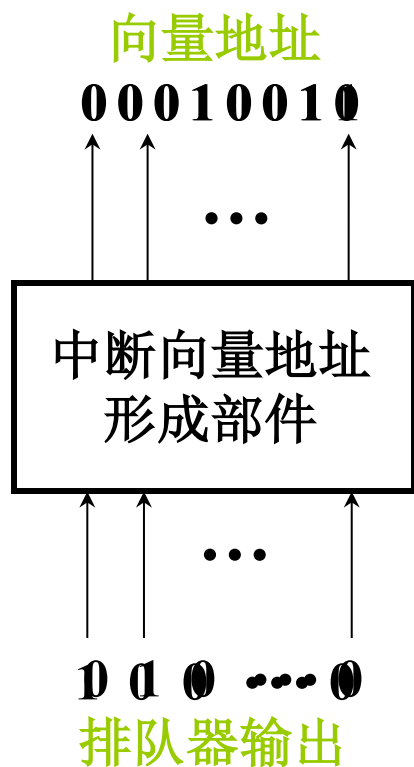
桔色表示信息为1，绿色为0，黑色任意，这里 $INTR_2$ 为1
表示有中断请求，排队器输出0100



3. 中断向量地址形成部件

- CPU响应I/O中断后，要暂停现执行程序，转去执行最高优先级设备的中断服务程序，因此首先要找到该设备服务程序的入口地址(中断服务程序首地址)
- 入口地址寻找方法包括硬件法和软件法（后续章节介绍），硬件向量法就是通过向量地址寻找设备中断服务程序入口地址，而向量地址由硬件电路产生
- 中断向量地址形成部件在排队器之后，排队器的输出（多路信号，只有1个是高电平）是地址形成部件的输入，排队器形成的输出是一串二进制代码，即中断向量，每个中断源对应一个向量地址，因此该部件是一个编码器

形成向量地址



设备编码器
(输出位数与中断源个数有关)

171=2 (1) 逻辑分析: 207PA=0 时, 207PA=1 时, 能送到 02

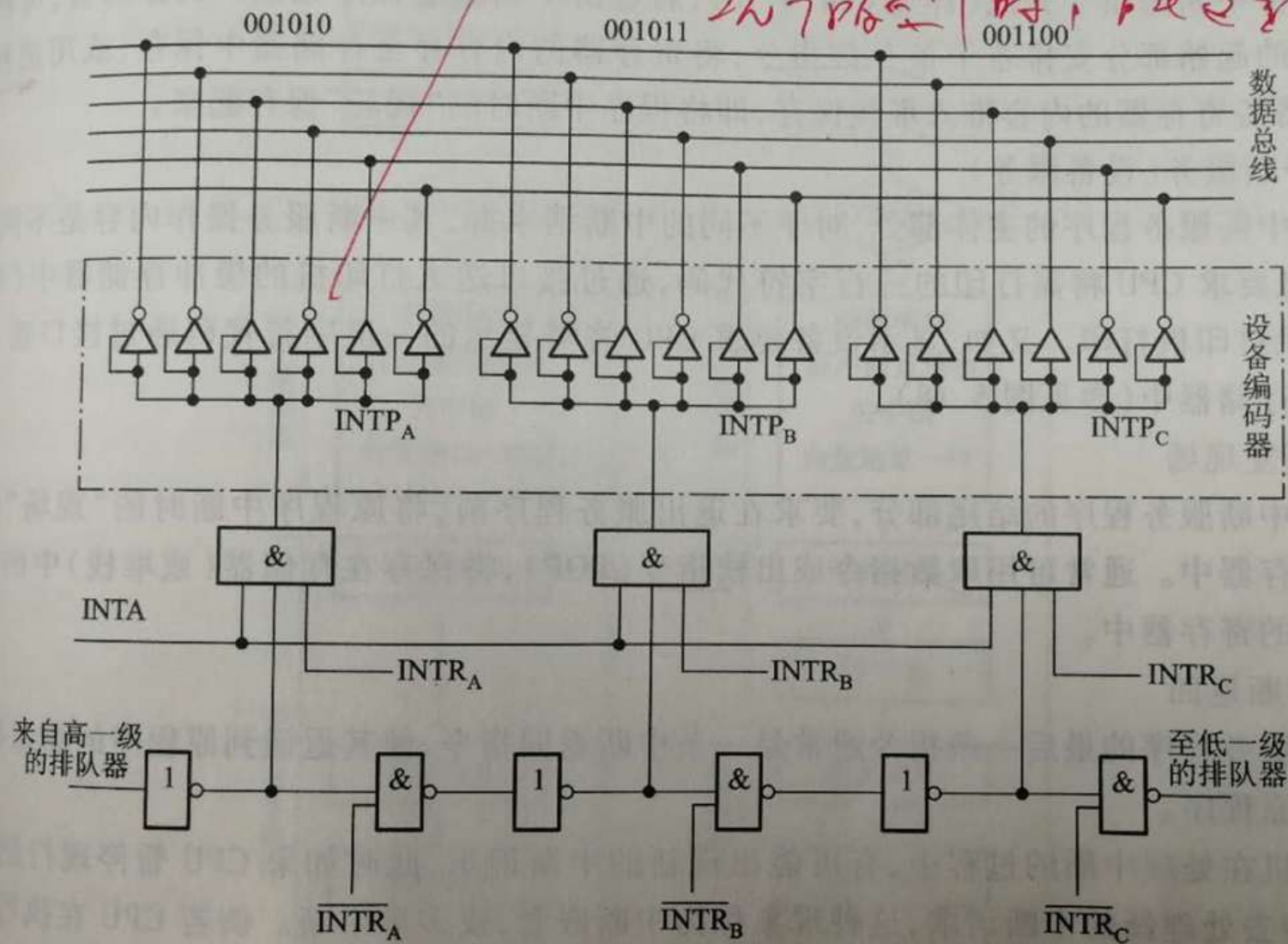
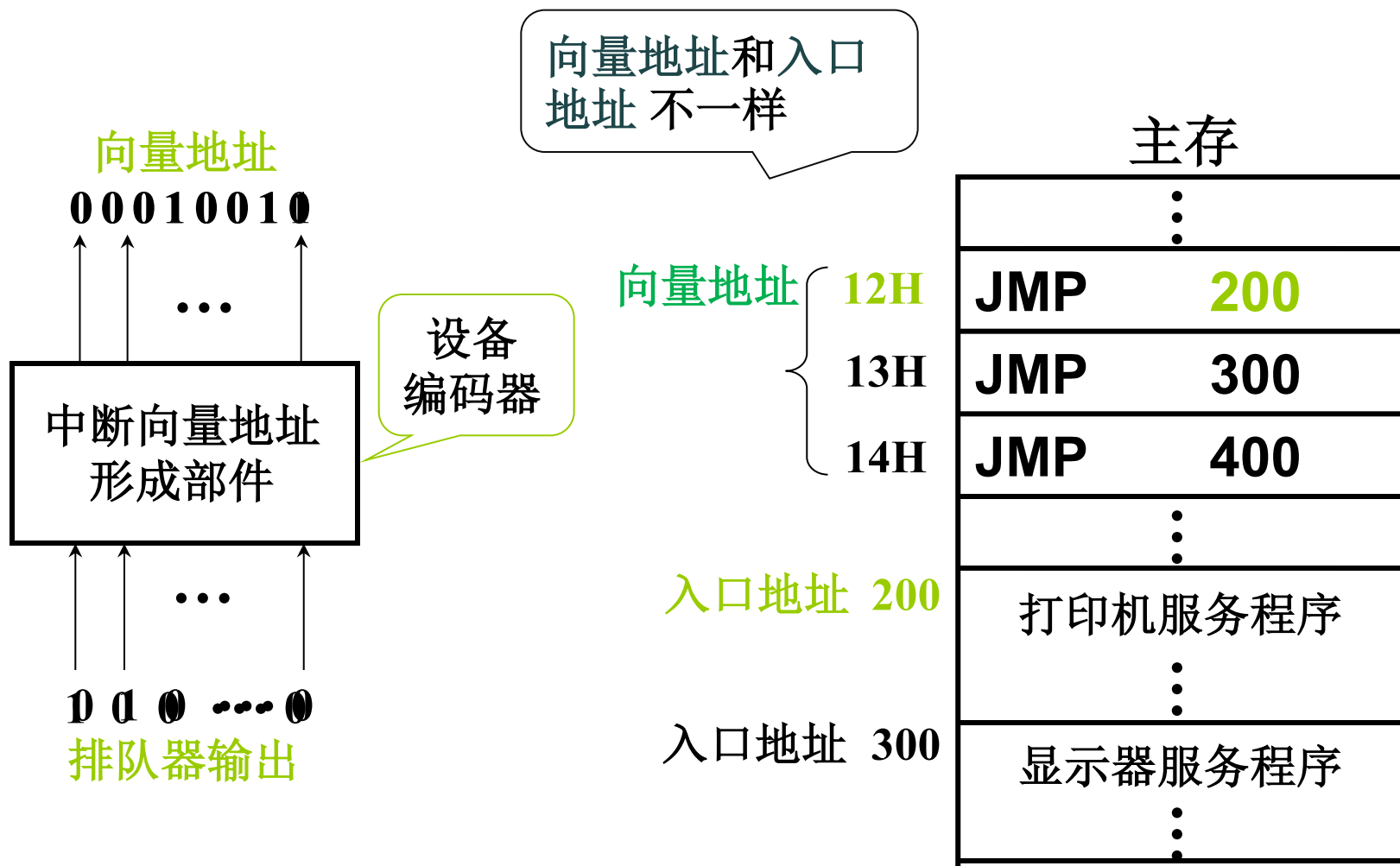


图 5.42 例 5.2 电路图

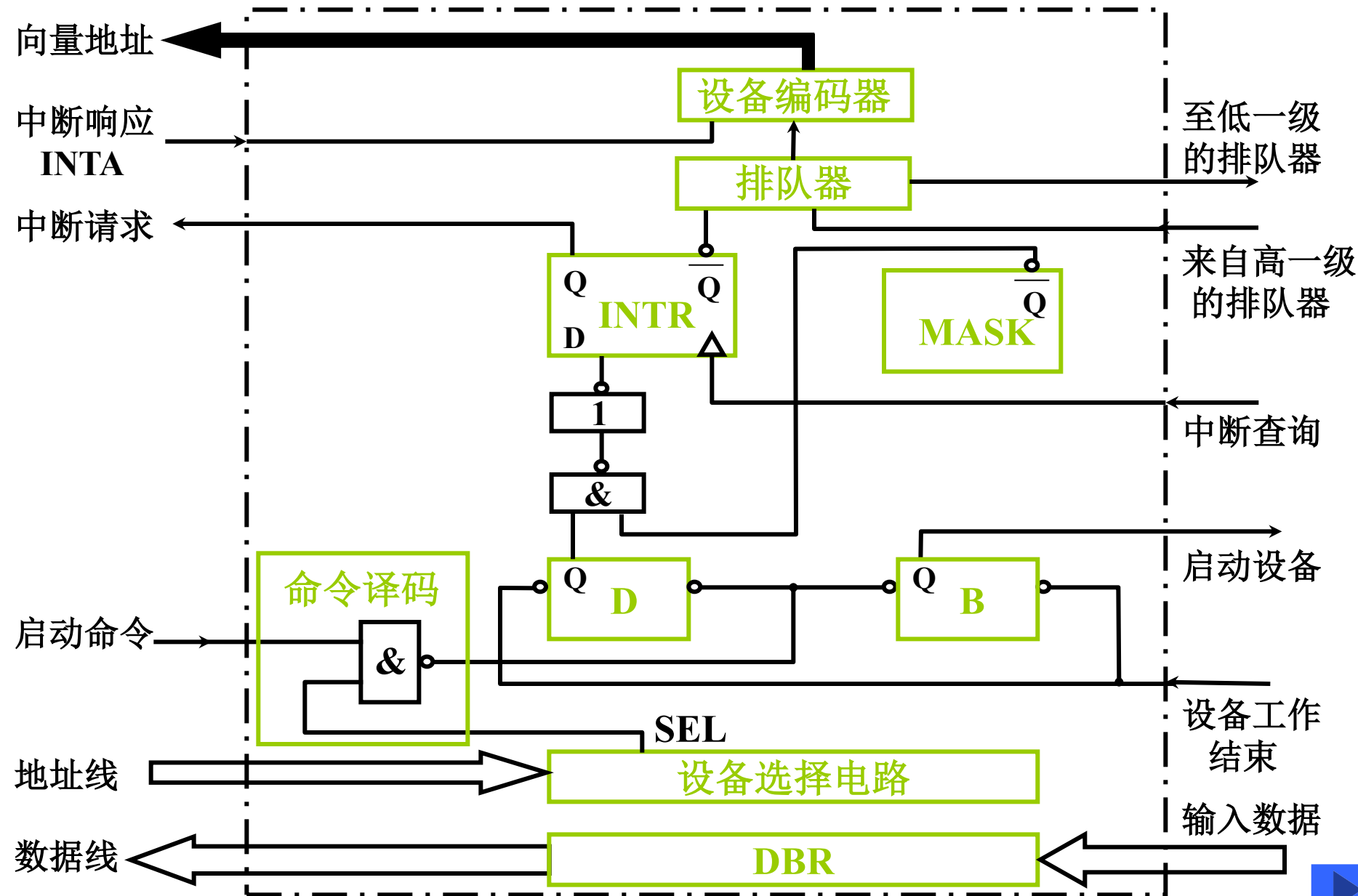
如何由向量地址找到入口地址

- 向量地址对应连续的内存单元---每个内存单元，存放一条跳转到某中断服务程序的指令，或者是保存某中断服务程序的入口地址
- 如何运行中断服务程序，需要把入口地址送到PC，有两种方法：
 - ① 向量地址送至PC，运行跳转指令
 - ② 向量地址对应的内存单元内容送至PC

从向量地址到入口地址



4. 程序中断方式接口电路的基本组成



四、I/O 中断处理过程

1. CPU 响应中断的条件和时间

(1) 条件：中断开关是打开的

为什么设置中断开关？
和中断屏蔽有啥不一样

即：CPU中的允许中断触发器 $EINT = 1$

用 开中断 指令将 $EINT$ 置 “1”

用 关中断 指令将 $EINT$ 置 “0” 或硬件自动复位

(2) 时间：收到中断请求时

即：在每条指令执行阶段的结束时刻

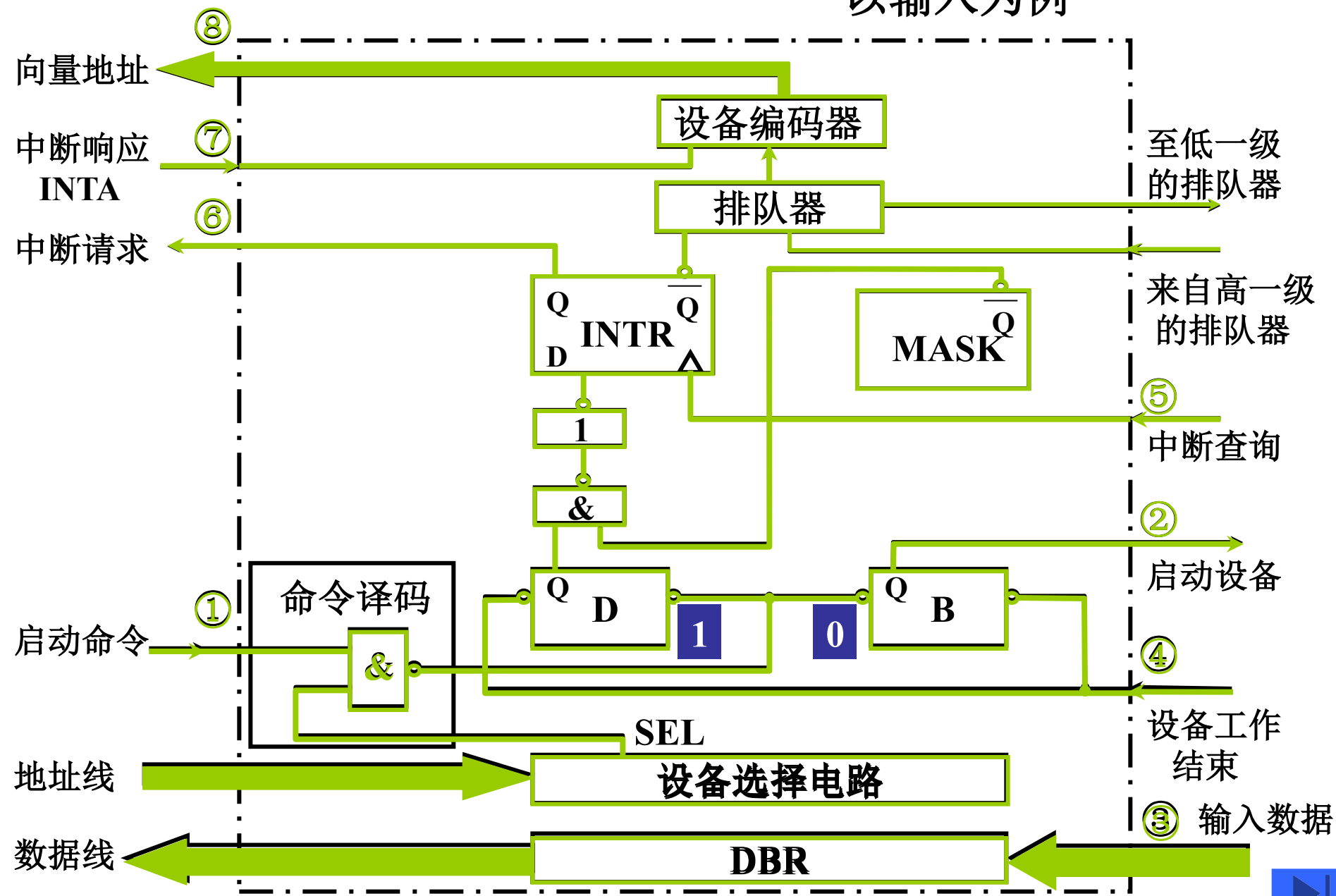
CPU 发 中断查询信号（将有下述情况的 $INTR$ 置 “1”）

当 $D = 1$ （何时设备就绪随机）且 $MASK = 0$ 时



2. I/O 中断处理过程

以输入为例



- ✓ 一次中断处理的完整过程包括如下阶段：中断请求、中断判优、中断响应、中断服务、中断返回（后两个环节软件实现）
- ✓ 中断服务程序最后一条指令是一条中断返回指令，用于从中断服务程序返回原程序断点

五、中断服务程序流程

1. 中断服务程序的流程

与进程切换一样，中断响应需要保护寄存器数据---保护现场

(1) 保护现场

- { 程序断点地址的保护 由中断隐指令（硬件）完成 why?
- { 寄存器内容（通用/状态）的保护 由进栈指令或者写存储器（软件）完成 why?

(2) 中断服务

对不同的 I/O 设备，有不同内容的中断服务程序

(3) 恢复现场

使用出栈或者读存储器指令，与保护时操作对应

(4) 中断返回

ISR 的最后一条指令通常是一条中断返回指令，返回到原程序断点处，继续执行原程序

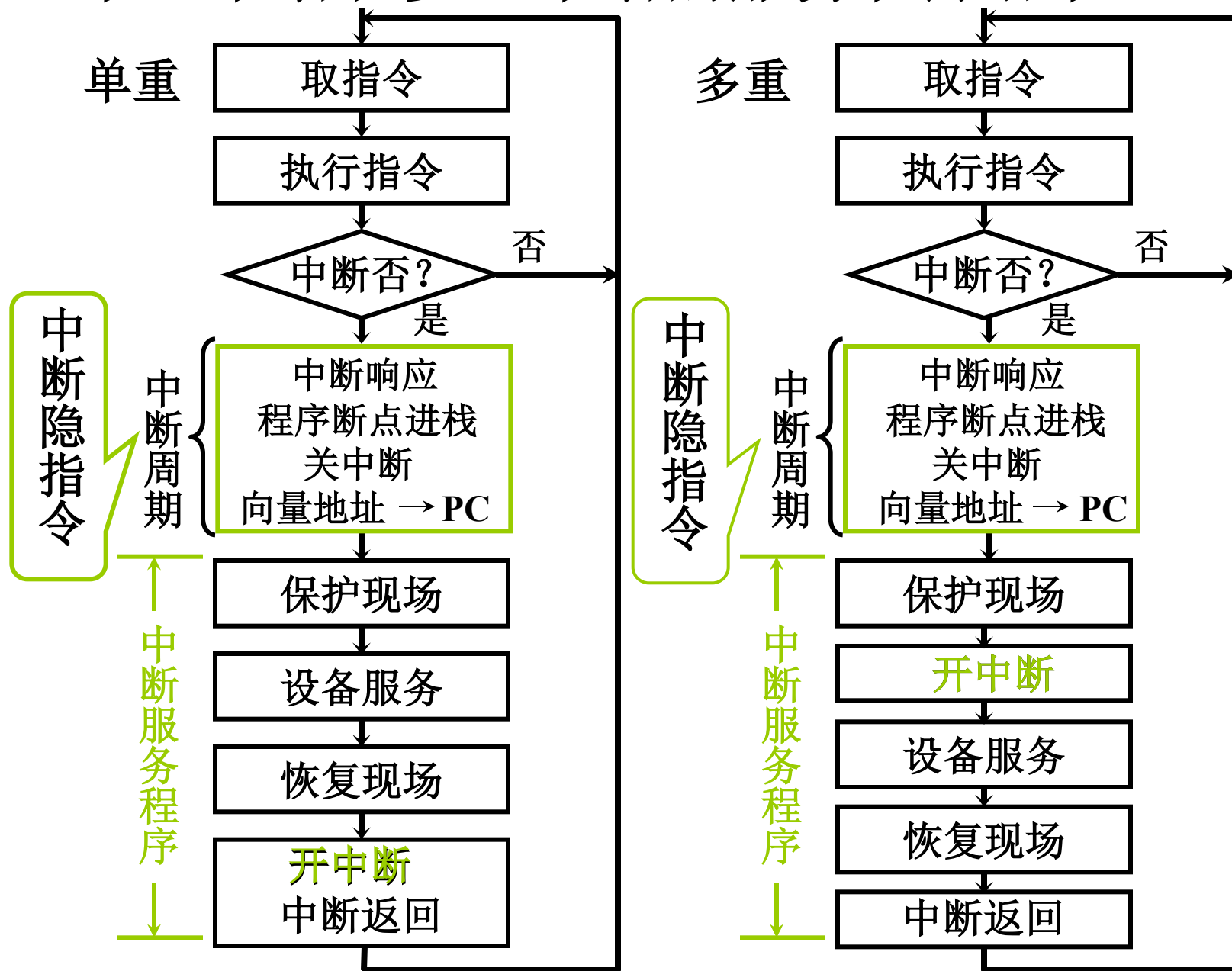
程序中断方式由于软件额外开销时间比较大，虽然 CPU 运行效率比程序查询高，但传输速度却比程序查询慢



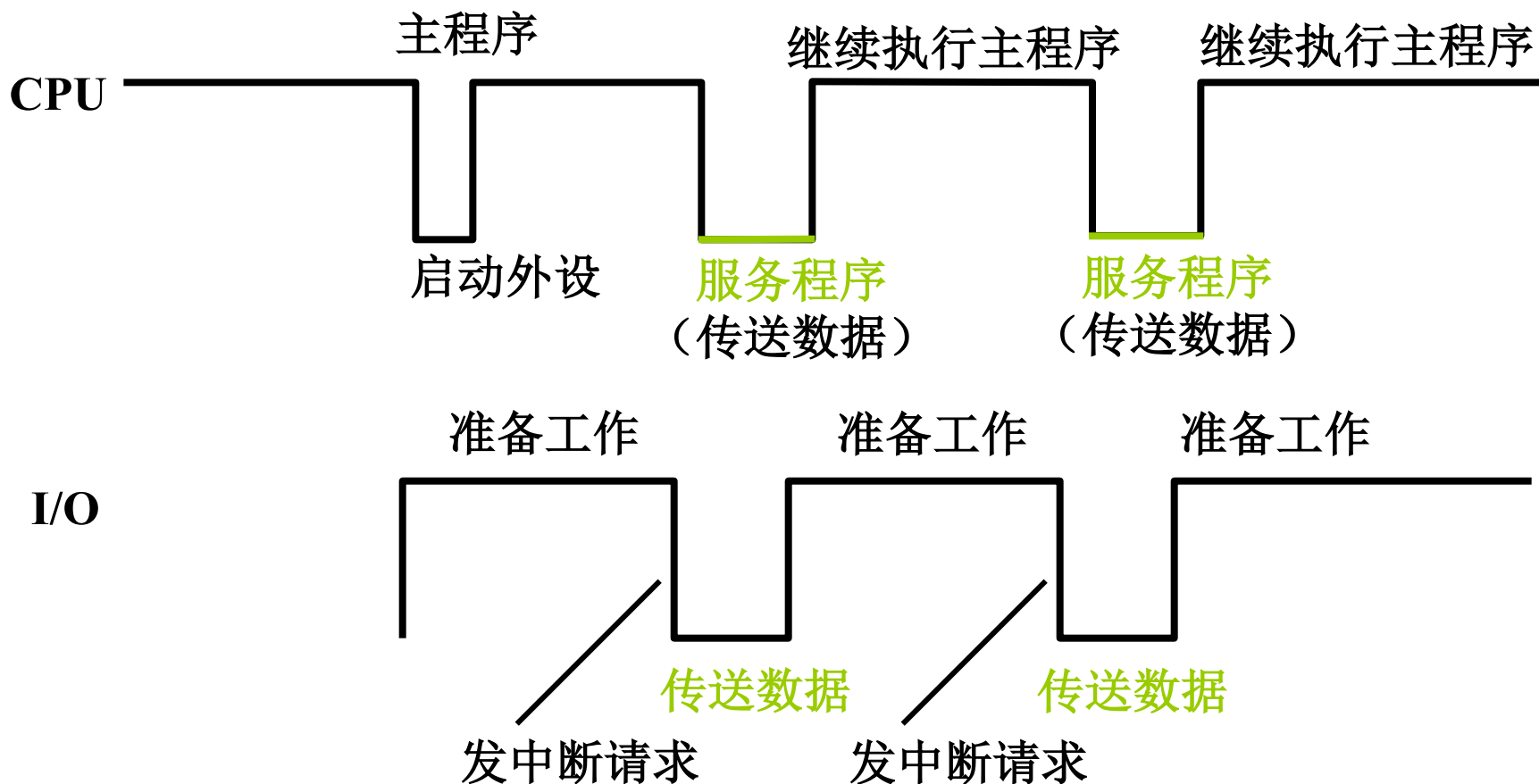
2. 单重中断和多重中断

- 计算机处理中断过程中，有可能出现新的中断请求，若CPU暂停执行现行中断服务程序，转去处理新的中断请求，称为多重中断或中断嵌套，若对新中断不予理睬，则为单重中断。
- 嵌套的前提：只有优先级更高的新中断才能中断当前中断服务程序，且只有在EINT=1时才可以。
- 为何有前提2：CPU响应中断后，硬件自动关中断，使EINT=0，若以后不用开中断指令把EINT置1，则CPU不能响应其他中断源的请求。
- 所以，单重中断和多重中断在中断服务程序流程方面存在区别：开中断的时刻不同。

3. 单重中断和多重中断的服务程序流程



主程序和服务程序抢占 CPU 示意图



宏观上CPU 和 I/O并行工作，微观上 CPU中断现执行程序为 I/O 服务，对于高速I/O设备，需要不断打断CPU 执行主程序以执行中断服务，效率低。

